

05_sensitivity_kNN

February 16, 2026

1 05 — Sensitivity to kNN parameter k

Purpose: Test sensitivity to k (number of neighbors in the kNN graph). Vary k {10, 20, 30, 50}, compute spatial metrics, plot stabilization curves.

1.1 Imports and path setup

```
[7]: import sys
from pathlib import Path

ROOT = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))
if str(ROOT / "src") not in sys.path:
    sys.path.insert(0, str(ROOT / "src"))

import warnings

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# kNN with small k can yield disconnected graph components; libpysal warns.
# ↳ Results are still valid.
warnings.filterwarnings("ignore", message=".*not fully connected.*",
    ↳category=UserWarning)

from analysis.experiment_runner import _get_run_dirs
from analysis.preprocessing import get_transformed_test_features
from analysis.run_analysis import load_meta, run_multiplicity, run_spatial

RESULTS_DIR = ROOT / "results"
```

1.2 Loop over k and store results

```
[8]: DATASET = "compas"
K = 25
K_NN_LIST = [10, 20, 30, 50]

dataset_dir = RESULTS_DIR / DATASET
run_dirs = _get_run_dirs(dataset_dir)

def run_one_knn(run_dir, X_test, k_nn):
    n_cand = len(load_meta(run_dir))
    K_actual = min(K, n_cand)
    mult = run_multiplicity(run_dir, K=K_actual)
    spatial = run_spatial(run_dir, X_test, K=K_actual, k=k_nn)
    return {"mean_variance": mult["mean_variance"], "moran_i":  
    ↪spatial["moran_i"], "n_hh": int(np.sum(spatial["HH_mask"]))}

results_knn = []
for k_nn in K_NN_LIST:
    for run_dir in run_dirs:
        X_test = get_transformed_test_features(run_dir, DATASET)
        res = run_one_knn(run_dir, X_test, k_nn)
        results_knn.append({"k_nn": k_nn, **res})

df_knn = pd.DataFrame(results_knn)
print(df_knn.groupby("k_nn").agg({"mean_variance": ["mean", "std"], "moran_i":  
    ↪["mean", "std"], "n_hh": "mean"}).round(4))
```

	mean_variance		moran_i		n_hh
	mean	std	mean	std	mean
k_nn					
10	0.0013	0.0003	0.2943	0.1146	40.3
20	0.0013	0.0003	0.2304	0.0837	68.8
30	0.0013	0.0003	0.1993	0.0772	80.7
50	0.0013	0.0003	0.1627	0.0646	115.2

1.3 Plot elbow curves

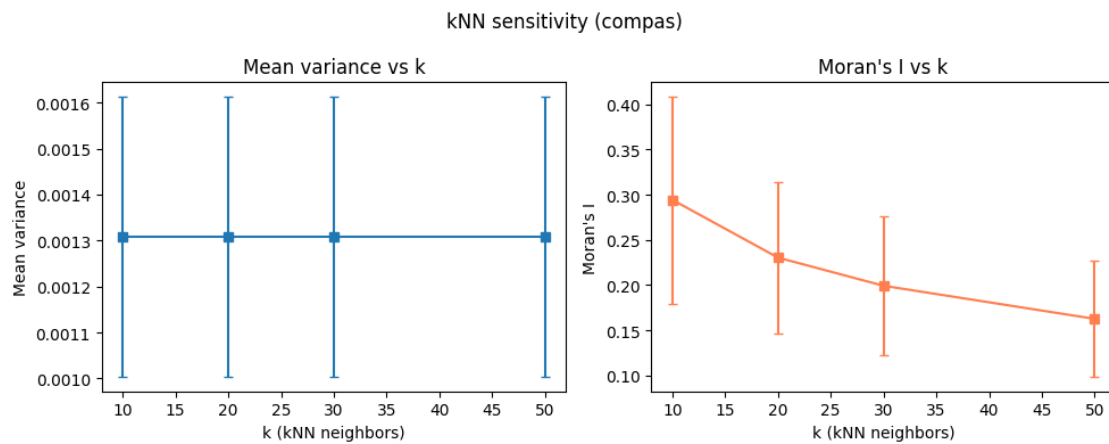
```
[9]: agg_knn = df_knn.groupby("k_nn").agg(
    mean_variance_mean=("mean_variance", "mean"),  
    ↪mean_variance_std=("mean_variance", "std"),  
    moran_mean=("moran_i", "mean"), moran_std=("moran_i", "std"),  
    n_hh_mean=("n_hh", "mean")).reset_index()

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 4))
ax1.errorbar(agg_knn["k_nn"], agg_knn["mean_variance_mean"],  
    ↪yerr=agg_knn["mean_variance_std"], marker="s", capsize=3)
ax1.set_xlabel("k (kNN neighbors)")
```

```

ax1.set_ylabel("Mean variance")
ax1.set_title("Mean variance vs k")
ax2.errorbar(agg_knn["k_nn"], agg_knn["moran_mean"], yerr=agg_knn["moran_std"],
            marker="s", capsize=3, color="coral")
ax2.set_xlabel("k (kNN neighbors)")
ax2.set_ylabel("Moran's I")
ax2.set_title("Moran's I vs k")
plt.suptitle(f"kNN sensitivity ({DATASET})")
plt.tight_layout()
plt.show()

```



1.4 Interpret stability

If spatial statistics change little for $k = 20-30$, the kNN graph choice is stable.